

Policy Gradient:

Reinforcement Learning - Lecture 12

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

- 1 Recap
- 2 Introduction
 - Examples
 - Policy Search
- 3 Monte-Carlo Policy Gradient
 - Likelihood Ratios
 - Policy Gradient Theorem
- 4 Actor-Critic Policy Gradient
 - Compatible Function Approximation
 - Advantage Function Critic
 - Eligibility Traces
- 5 Summary and Conclusion

- Last lecture:
 - On-policy approximations
 - Approximation-based Prediction and Control
- This lecture (Sutton and Barto 13.1-13.4)
 - **Policy Gradient Methods**
 - Actor-Critic architectures
- Next lecture
 - **Deep Reinforcement Learning**
 - Example applications

Recap

- We expanded states from monolithic entries in a table into a **factored representation** of the states
- This allowed use to learn parameters for **approximate** value functions
- Parameterizing the value function allows better scalability

Outline

- 1 Recap
- 2 Introduction
 - Examples
 - Policy Search
- 3 Monte-Carlo Policy Gradient
 - Likelihood Ratios
 - Policy Gradient Theorem
- 4 Actor-Critic Policy Gradient
 - Compatible Function Approximation
 - Advantage Function Critic
 - Eligibility Traces
- 5 Summary and Conclusion

Policy-Based Reinforcement Learning

- In the last lecture we approximated the value or action-value function using parameters $\mathbf{w}$,

$$\begin{aligned}V_{\theta}(s) &= \hat{v}_{\pi}(s, \mathbf{w}) \approx v_{\pi}(s) \\ Q_{\theta}(s, a) &= \hat{q}_{\pi}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)\end{aligned}$$

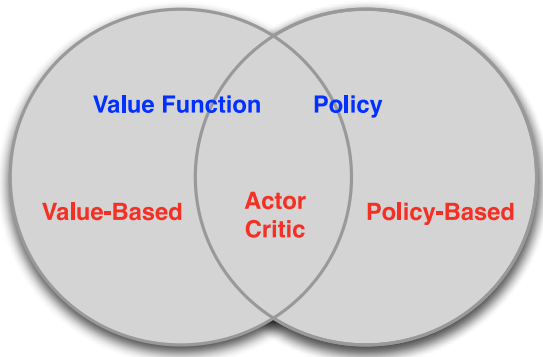
- We generated a policy directly from the value function
e.g. using ϵ -greedy
- In this lecture we will directly parameterize the **policy**

$$\pi_{\theta}(s, a) = \mathbb{P}[a \mid s, \theta]$$

- We will focus again on **model-free** reinforcement learning

Value-Based and Policy-Based RL

- Value Based
 - Learned Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learned Policy
- Actor Critic
 - Learned Value Function
 - Learned Policy



Advantages of Policy-Based RL

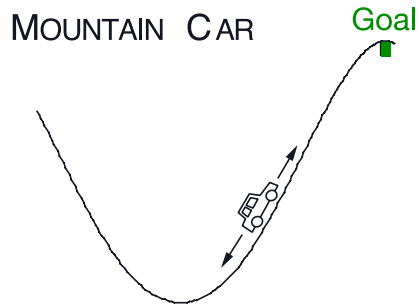
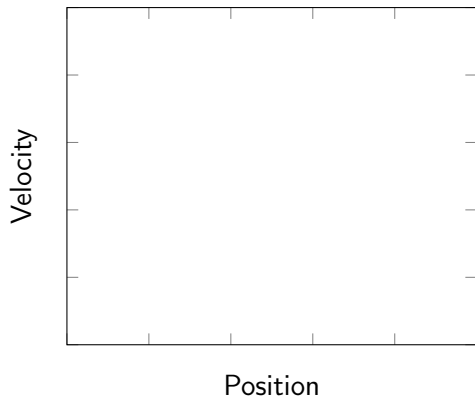
Advantages

- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

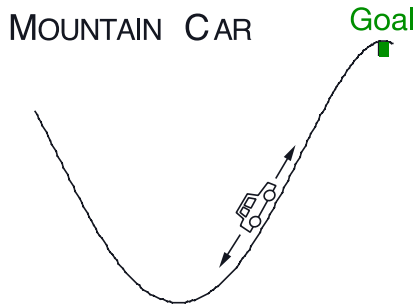
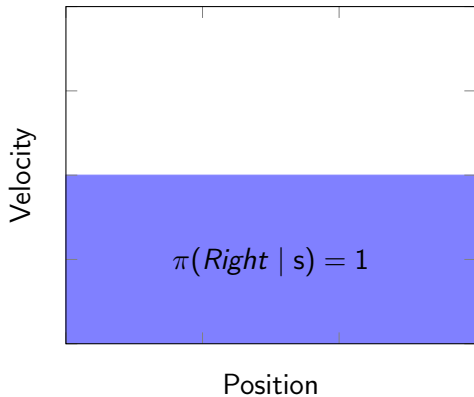
Disadvantages

- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance

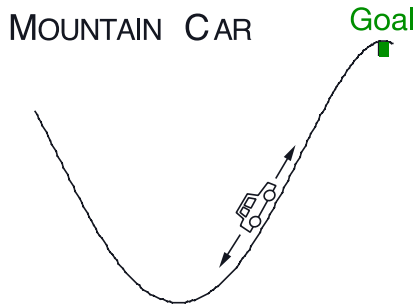
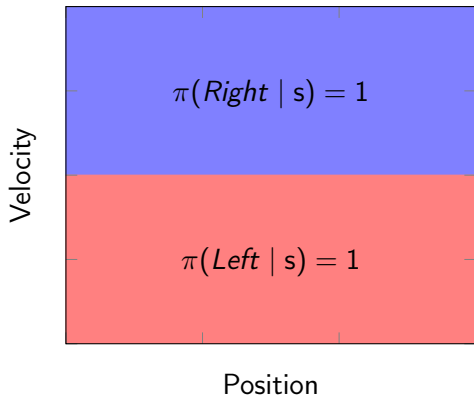
Example: Policy without Action-Values



Example: Policy without Action-Values



Example: Policy without Action-Values

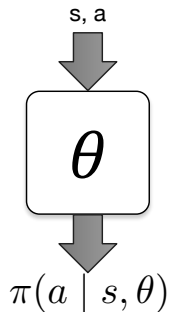
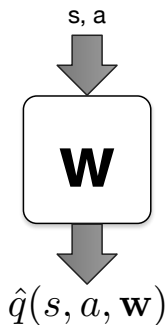


Example: Rock-Paper-Scissors



- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for **iterated** rock-paper-scissors
 - A deterministic policy is easily exploited
 - A uniform random policy is optimal (i.e. Nash equilibrium)

Parameterizing Policies Directly



- w – parameters of a value function
- θ – parameters of a policy

Constraints on Policy Parameterization

$$\pi(a \mid s, \theta) \geq 0$$

for all $a \in \mathcal{A}$ **and** $s \in \mathcal{S}$

$$\sum_{a \in \mathcal{A}} \pi(a \mid s, \theta) = 1$$

for all $s \in \mathcal{S}$

The Softmax Policy Parameterization

$$\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\boldsymbol{\theta})}}$$

where, e.g. $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(s, a)$

The Softmax Policy Parameterization

$$\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{\overbrace{e^{h(s,a,\boldsymbol{\theta})}}^{\text{Action Preference}}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\boldsymbol{\theta})}}$$

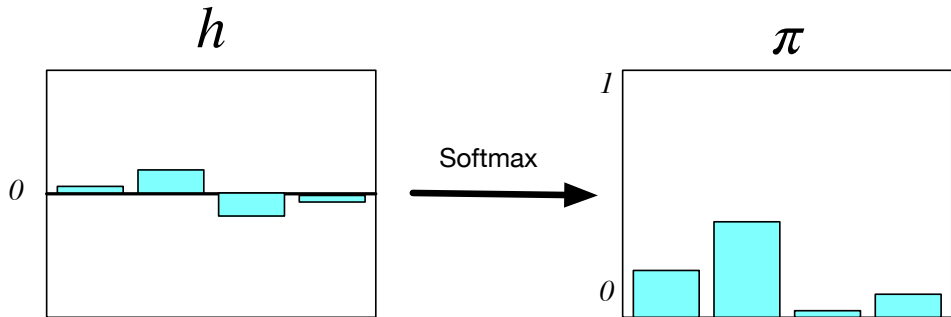
where, e.g. $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(s, a)$

The Softmax Policy Parameterization

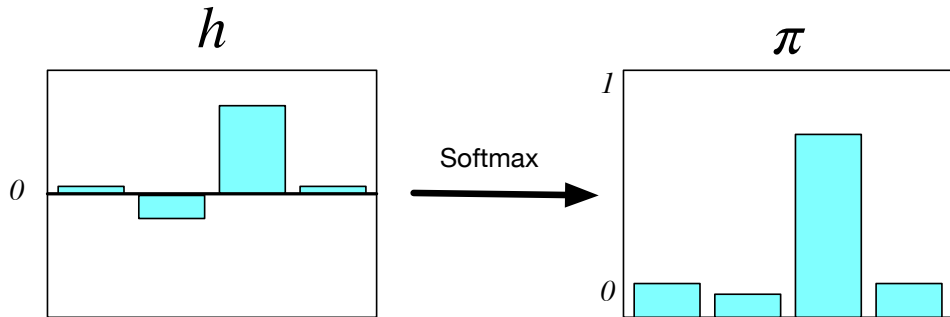
$$\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{\overbrace{e^{h(s,a,\boldsymbol{\theta})}}^{\text{Action Preference}}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\boldsymbol{\theta})}}$$

where, e.g. $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(s, a)$
or a neural network

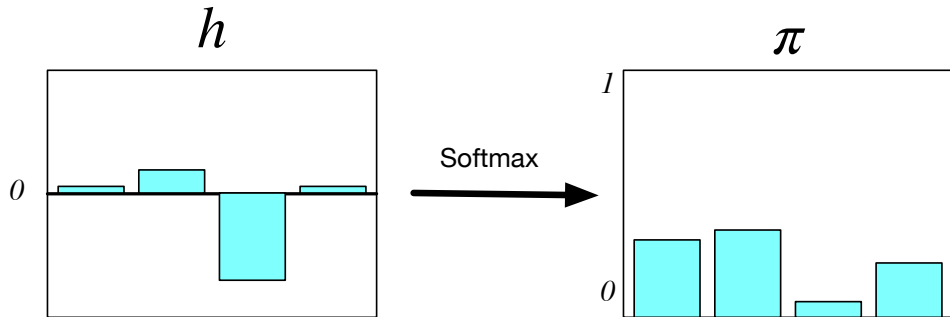
Examples of Softmax Policies



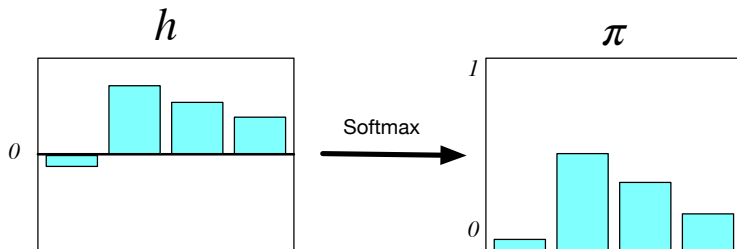
Examples of Softmax Policies



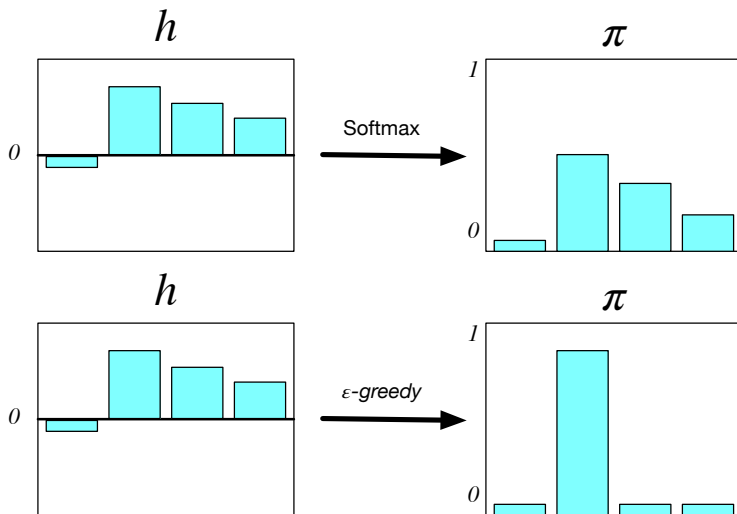
Examples of Softmax Policies



Action preferences are not action values

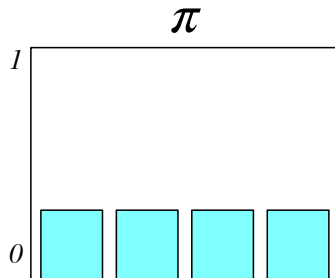


Action preferences are not action values



The Flexibility of Stochastic Policies

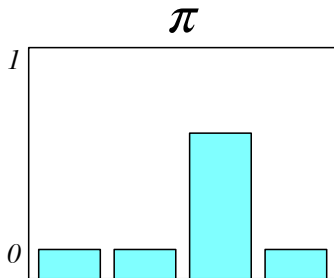
$$\pi(a \mid s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$



- Stochastic policies allow us to modulate exploration naturally
- Suitable for when optimal policy is stochastic

The Flexibility of Stochastic Policies

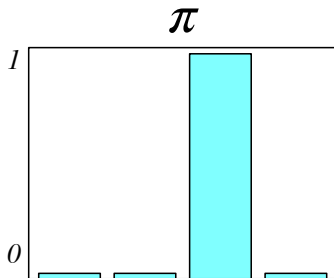
$$\pi(a \mid s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$



- Stochastic policies allow us to modulate exploration naturally
- Suitable for when optimal policy is stochastic

The Flexibility of Stochastic Policies

$$\pi(a \mid s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$

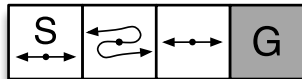


- Stochastic policies allow us to modulate exploration naturally
- Suitable for when optimal policy is stochastic

The Flexibility of Stochastic Policies (2)

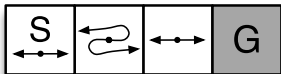
Stochastic policies might be better than deterministic policies under function approximation.

$$\begin{aligned}\pi_* : \mathcal{S} &\mapsto \mathcal{A} \\ \pi_*(s_1) &= a_1 \\ \pi_*(s_2) &= a_2\end{aligned}$$



Example: Stochastic Policies

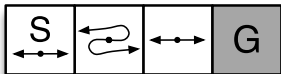
Consider the environment:



$$\mathcal{R} = -1$$

Example: Stochastic Policies

Consider the environment:



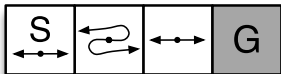
$$\mathcal{R} = -1$$

Deterministic policy

$$v_{\pi}(S) = -\infty$$

Example: Stochastic Policies

Consider the environment:



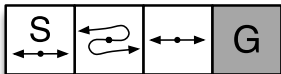
$$\mathcal{R} = -1$$

Deterministic policy

$$v_{\pi}(S) = -\infty$$

Example: Stochastic Policies

Consider the environment:



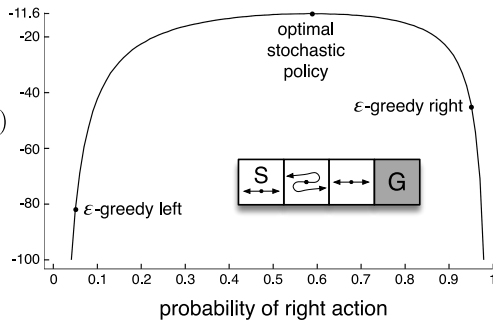
$$\mathcal{R} = -1$$

Deterministic policy

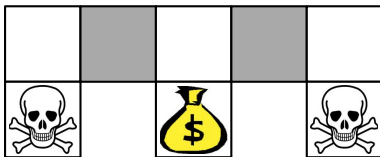
$$v_{\pi}(S) = -\infty$$

$$\pi(\text{right} \mid S) = 0.59$$

$$J(\theta) = v_{\pi_{\theta}}(S)$$



Example: Aliased Gridworld (1)



- The agent cannot differentiate the grey states
- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = 1(\text{wall to } N, a = \text{move } E)$$

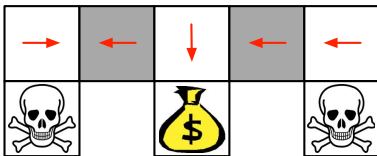
- Compare value-based RL, using an approximate value function

$$Q_{\theta}(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parameterized policy

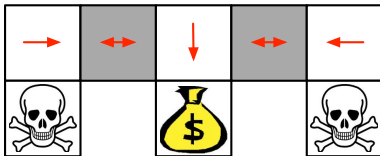
$$\pi_{\theta}(a \mid s) = g(\phi(s, a), \theta)$$

Example: Aliased Gridworld (2)



- Under aliasing, an optimal **deterministic** policy will either
 - move *W* in both grey states (shown by red arrows)
 - move *E* in both grey states
- Either way, it can get stuck and **never** reach the money
- Value-based RL learns a near-deterministic policy
 - e.g. greedy or ϵ -greedy
- So it will traverse the corridor for a long time

Example: Aliased Gridworld (3)



- An optimal **stochastic** policy will randomly move E or W in grey states

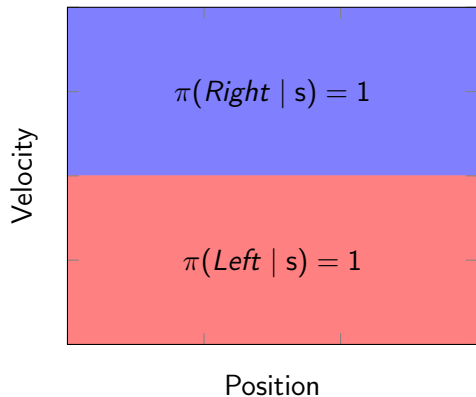
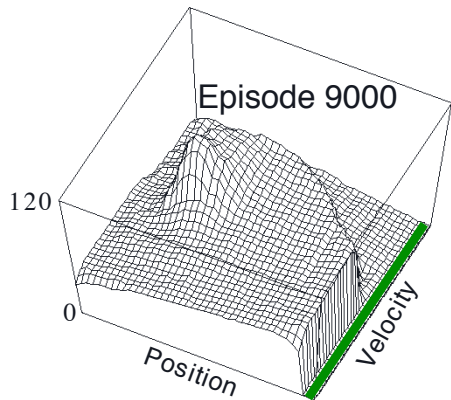
$$\pi_{\theta}(\text{wall to N and S, move } E) = 0.5$$

$$\pi_{\theta}(\text{wall to N and S, move } W) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn the optimal stochastic policy

Stochastic Policies

Sometimes it is easier to learn a good policy



Formalizing Goal as an Objective

RL Goal: Maximize Reward in the Long Run: $R_t, R_{t+1}, R_{t+2}, \dots$

Formalizing the Goal as an objective:

Episodic:
$$G_t = \sum_{t=0}^T R_t$$

Continuing:
$$G_t = \sum_{t=0}^{\infty} \gamma^t R_t \quad G_t = \sum_{t=0}^{\infty} R_t - r(\pi)$$

The Average Reward Objective

$$r(\pi)$$

The Average Reward Objective

$$r(\pi) = \sum_s \mu(s) \sum_a \pi(a \mid s, \theta) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

The Average Reward Objective

$$r(\pi) = \sum_s \mu(s) \sum_a \pi(a \mid s, \theta) \underbrace{\sum_{s', r} \mathcal{P}(s', r \mid s, a) r}_{\mathbb{E}[R_t \mid S_t=s, A_t=a]}$$

The Average Reward Objective

$$r(\pi) = \sum_s \mu(s) \underbrace{\sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r}_{\mathbb{E}[R_t | S_t=s]}$$

The Average Reward Objective

$$r(\pi) = \underbrace{\sum_s \mu(s) \sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r}_{\mathbb{E}[R_t]}$$

Optimizing the Average Reward Objective

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_{\mathbf{s}} \mu(\mathbf{s}) \sum_{\mathbf{a}} \pi(\mathbf{a} \mid \mathbf{s}, \boldsymbol{\theta}) \sum_{\mathbf{s}', r} \mathcal{P}(\mathbf{s}', r \mid \mathbf{s}, \mathbf{a}) r$$

Policy-Gradient Method

Challenge of Policy Gradient Methods

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_{\mathbf{s}} \mu(\mathbf{s}) \sum_{\mathbf{a}} \pi(\mathbf{a} \mid \mathbf{s}, \boldsymbol{\theta}) \sum_{\mathbf{s}', r} \mathcal{P}(\mathbf{s}', r \mid \mathbf{s}, \mathbf{a}) r$$

Challenge of Policy Gradient Methods

$$\nabla_{\theta} r(\pi) = \nabla_{\theta} \sum_s \mu(s) \sum_s \pi(a \mid s, \theta) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

Instead of minimizing error, we want to maximize average reward

Challenge of Policy Gradient Methods

$$\nabla_{\theta} r(\pi) = \nabla_{\theta} \sum_s \underbrace{\mu(s)}_{\text{Depends on } \theta} \sum_s \pi(a \mid s, \theta) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

Challenge of Policy Gradient Methods

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_s \underbrace{\mu(s)}_{\text{Depends on } \boldsymbol{\theta}} \sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

$$\nabla_{\mathbf{w}} \overline{VE} = \nabla_{\mathbf{w}} \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

Challenge of Policy Gradient Methods

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_s \underbrace{\mu(s)}_{\text{Depends on } \boldsymbol{\theta}} \sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

$$\nabla_{\boldsymbol{w}} \overline{VE} = \nabla_{\boldsymbol{w}} \sum_s \underbrace{\mu(s)}_{\text{Independent of } \boldsymbol{w}} [v_{\pi}(s) - \hat{v}(s, \boldsymbol{w})]^2$$

Challenge of Policy Gradient Methods

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_s \underbrace{\mu(s)}_{\text{Depends on } \boldsymbol{\theta}} \sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

$$\nabla_{\mathbf{w}} \overline{VE} = \nabla_{\mathbf{w}} \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

Challenge of Policy Gradient Methods

$$\nabla_{\boldsymbol{\theta}} r(\pi) = \nabla_{\boldsymbol{\theta}} \sum_s \underbrace{\mu(s)}_{\text{Depends on } \boldsymbol{\theta}} \sum_s \pi(a \mid s, \boldsymbol{\theta}) \sum_{s', r} \mathcal{P}(s', r \mid s, a) r$$

$$\begin{aligned} \nabla_{\mathbf{w}} \overline{VE} &= \nabla_{\mathbf{w}} \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= \sum_s \mu(s) \nabla_{\mathbf{w}} [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \end{aligned}$$

Policy Objective Functions

- Goal: given policy $\pi_{\theta}(s, a)$ with parameters θ , find best θ
- But how do we measure the quality of a policy π_{θ} ?
- In episodic environments we can use the **start value**

$$J_1(\theta) = V_{\pi_{\theta}}(s_1) = \mathbb{E}_{\pi_{\theta}}[v_1]$$

- In continuing environments we can use the **average value**

$$J_{avV}(\theta) = \sum_s \mu_{\pi_{\theta}}(s) V_{\pi_{\theta}}(s)$$

- Or the **average reward per time-step**

$$J_{avR}(\theta) = \sum_s \mu_{\pi_{\theta}}(s) \sum_a \pi(a | s, \theta) \mathcal{R}_s^a$$

- where $\mu_{\pi_{\theta}}(s)$ is the **stationary distribution** of Markov chain for π_{θ}

Policy Optimization

- Policy based reinforcement learning is an **optimization** problem
- Find θ that maximizes $J(\theta)$
- Some approaches do not use gradient
 - Hill climbing
 - Simplex / amoeba / Nelder Mead
 - Genetic algorithms
- Greater efficiency often possible using gradient
 - Gradient descent
 - Conjugate gradient
 - Quasi-newton
- We focus on gradient descent, many extensions possible
- And on methods that exploit sequential structure

Outline

- 1 Recap
- 2 Introduction
 - Examples
 - Policy Search
- 3 Monte-Carlo Policy Gradient
 - Likelihood Ratios
 - Policy Gradient Theorem
- 4 Actor-Critic Policy Gradient
 - Compatible Function Approximation
 - Advantage Function Critic
 - Eligibility Traces
- 5 Summary and Conclusion

Score Function

- We now compute the policy gradient analytically
- Assume policy π_{θ} is differentiable whenever it is non-zero and we know the gradient $\nabla_{\theta}\pi_{\theta}(a | s)$
- Likelihood ratios exploit the **following identity**

$$\begin{aligned}\nabla_{\theta}\pi_{\theta}(a | s) &= \pi_{\theta}(a | s) \frac{\nabla_{\theta}\pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} \\ &= \pi_{\theta}(a | s) \nabla_{\theta} \ln \pi_{\theta}(a | s)\end{aligned}$$

- The **score function** is $\nabla_{\theta} \ln \pi_{\theta}(a | s)$

Softmax Policy

- We will use a softmax policy as a running example
- Weight actions using linear combination of features $\phi(s, a)^\top \theta$
- Probability of action is proportional to exponentiated weight

$$\pi_\theta(a \mid s) \propto e^{\phi(s, a)^\top \theta}$$

- The score function is

$$\nabla_\theta \ln \pi_\theta(a \mid s) = \phi(s, a) - \mathbb{E}_{\pi_\theta} [\phi(s, \cdot)]$$

Gaussian Policy

- In continuous action spaces, a Gaussian policy is natural
- Mean is a linear combination of state features $\mu(s) = \phi(s)^\top \theta$
- Variance may be fixed σ^2 , or can also be parameterized
- Policy is Gaussian, $a \sim \mathcal{N}(\mu(s), \sigma^2)$
- The score function is

$$\nabla_{\theta} \ln \pi_{\theta}(a | s) = \frac{(a - \mu(s))\phi(s)}{\sigma^2}$$

One-Step MDPs

- Consider a simple class of **one-step** MDPs
 - Starting in state $s \sim \mu(s)$
 - Terminating after one time-step with reward $r = \mathcal{R}_{s,a}$
- Use likelihood ratios to compute the policy gradient

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [r] \\ &= \sum_{s \in \mathcal{S}} \mu(s) \sum_{a \in \mathcal{A}} \pi_{\theta}(a | s) \mathcal{R}_{s,a} \\ \nabla_{\theta} J(\theta) &= \sum_{s \in \mathcal{S}} \mu(s) \sum_{a \in \mathcal{A}} \pi_{\theta}(a | s) \nabla_{\theta} \ln \pi_{\theta}(a | s) \mathcal{R}_{s,a} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a | s) r] \end{aligned}$$

Policy Gradient Theorem

- The policy gradient theorem generalizes the likelihood ratio approach to multi-step MDPs
- Replaces instantaneous reward r with long-term value $q_\pi(s, a)$
- Policy gradient theorem applies to start state objective, average reward and average value objective

Theorem

*For any differentiable policy $\pi_\theta(a | s)$,
for any of the policy objective functions $J = J_1, J_{av}R$, or $\frac{1}{1-\gamma} J_{av}V$,
the policy gradient is*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \ln \pi_\theta(a | s) q_{\pi_\theta}(s, a)]$$

Monte-Carlo Policy Gradient (REINFORCE)

- Update parameters by stochastic gradient ascent
- Using policy gradient theorem
- Using return G_t as an unbiased sample of $q_{\pi\theta}(s_t, a_t)$

REINFORCE Algorithm, for estimating $\pi_{\theta} \approx \pi_*$

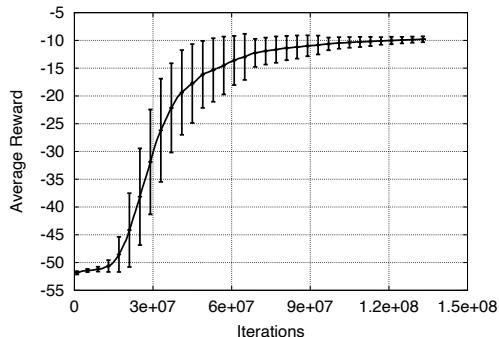
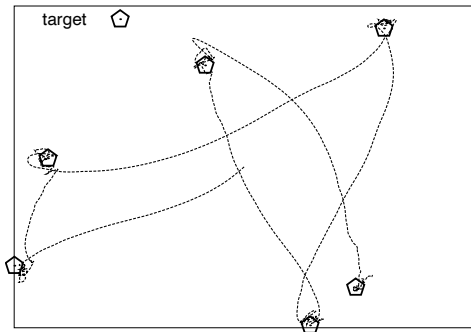
Require: a differentiable policy parameterization $\pi(a \mid s, \theta)$

Require: step size $\alpha > 0$

Require: Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g. to 0)

```
1: function REINFORCE( $\alpha$ )
2:   loop forever (for each episode)
3:     Generate an episode  $\{S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T\}$ , following  $\pi(\cdot \mid \cdot, \theta)$ 
4:     loop for each step in the episode  $t = 0, 1, \dots, T - 1$ 
5:        $G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$   $\triangleright G_t$ 
6:        $\theta \leftarrow \theta + \alpha \gamma^t G \nabla_{\theta} \ln \pi(A_t \mid S_t, \theta)$ 
```

Puck World Example



- Continuous actions exert small force on puck
- Puck is rewarded for getting close to target
- Target location is reset every 30 seconds
- Policy is trained using a variant of Monte-Carlo policy gradient

REINFORCE with Baseline (1)

- We can generalize the policy gradient theorem to include a comparison to a baseline $b(s)$

$$\nabla_{\theta} J(\theta) \propto \sum_s \mu(s) \sum_a q(s, a) \nabla \pi_{\theta}(a | s, \theta)$$

- The baseline can be any function, as long as it does not depend on an action a (e.g. it could be zero)
- A good baseline can diminish variance during learning, e.g. an estimate of the state value function $\hat{v}(s, \mathbf{w})$

REINFORCE with Baseline (1)

- We can generalize the policy gradient theorem to include a comparison to a baseline $b(s)$

$$\nabla_{\theta} J(\theta) \propto \sum_s \mu(s) \sum_a (q(s, a) - b(s)) \nabla \pi_{\theta}(a \mid s, \theta)$$

- The baseline can be any function, as long as it does not depend on an action a (e.g. it could be zero)
- A good baseline can diminish variance during learning, e.g. an estimate of the state value function $\hat{v}(s, \mathbf{w})$

Monte-Carlo Policy Gradient (REINFORCE with Baseline)

REINFORCE Algorithm with baseline, for estimating $\pi\theta \approx \pi_*$

Require: a differentiable policy parameterization $\pi(a | s, \theta)$

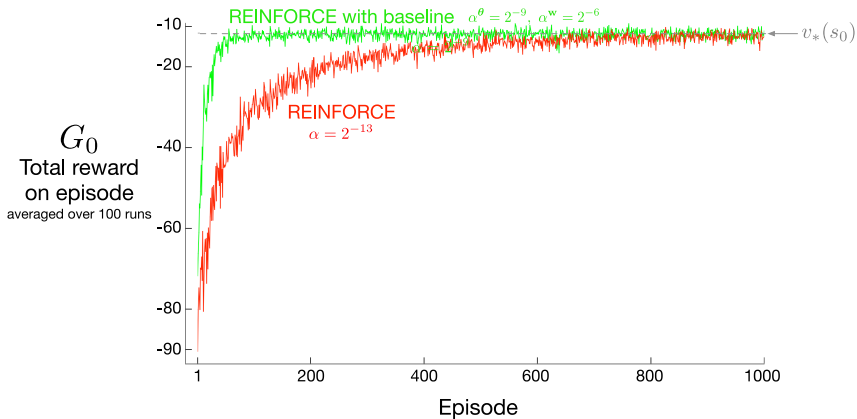
Require: a differentiable state-value function parameterization $\hat{v}(s, \theta)$

Require: step sizes $\alpha^\theta > 0$, $\alpha^w > 0$

Require: Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g. to 0)

- 1: **function** REINFORCEB(α^θ, α^w)
- 2: **loop** forever (for each episode)
- 3: Generate an episode $\{S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T\}$, following $\pi(\cdot | \cdot, \theta)$
- 4: **loop** for each step in the episode $t = 0, 1, \dots, T - 1$
- 5: $G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$ $\triangleright G_t$
- 6: $\delta \leftarrow G - \hat{v}(S_t, w)$
- 7: $w \leftarrow w + \alpha^w \delta \nabla_w \hat{v}(S_t, w)$
- 8: $\theta \leftarrow \theta + \alpha \gamma^t \delta \nabla_\theta \ln \pi(A_t | S_t, \theta)$

Gridworld Example



Outline

- 1 Recap
- 2 Introduction
 - Examples
 - Policy Search
- 3 Monte-Carlo Policy Gradient
 - Likelihood Ratios
 - Policy Gradient Theorem
- 4 Actor-Critic Policy Gradient
 - Compatible Function Approximation
 - Advantage Function Critic
 - Eligibility Traces
- 5 Summary and Conclusion

Reducing Variance Using a Critic

- Monte-Carlo policy gradient still has high variance
- We use a **critic** to estimate the action-value function,

$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi\theta}(s, a)$$

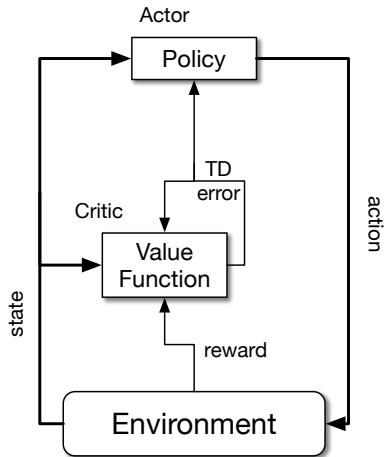
- Actor-critic algorithms maintain two sets of parameters
 - Critic** Updates action-value function parameters $\mathbf{w}$
 - Actor** Updates policy parameters θ , in direction suggested by critic
- Actor-critic algorithms follow an **approximate** policy gradient

$$\begin{aligned}\nabla_{\theta} J(\theta) &\approx \mathbb{E}_{\pi\theta} [\nabla_{\theta} \ln \pi(a | s, \theta) \hat{q}(s, a, \mathbf{w})] \\ \Delta \theta &= \alpha \nabla_{\theta} \ln \pi(a | s, \theta) \hat{q}(s, a, \mathbf{w})\end{aligned}$$

Estimating the Action-Value Function

- The critic is solving a familiar problem: policy evaluation
- How good is policy π_{θ} for current parameters θ
- This problem was explored in previous lectures, e.g.
 - Monte-Carlo policy evaluation
 - Temporal-Difference learning
 - TD(λ)
- Could also use e.g. least-squares policy evaluation

How the actor and the critic interact



$$\theta_{t+1} \doteq \theta_t + \alpha \nabla_{\theta} \ln \pi(A_t | S_t, \theta_t) \delta_t$$

Action-Value Actor-Critic

- Simple actor-critic algorithm based on action-value critic
- Using linear value function approximation $\hat{q}(s, a, \mathbf{w}) = \phi(s, a)^\top \mathbf{w}$

Critic Updates $\mathbf{w}$ by linear TD(0)

Actor Updates θ by policy gradient

Simple Q-Value Actor-Critic

```
1: function QAC( $\alpha^\theta, \alpha^w$ )
2:   Initialize  $S, \theta$ 
3:   Sample  $A \sim \pi(A \mid S, \theta)$ 
4:   for each step do
5:     Sample reward  $R = \mathcal{R}_S^A$ 
6:     Sample transition  $S' \sim \mathcal{P}(\cdot \mid S, A)$ 
7:     Sample  $A' \sim \pi(A' \mid S', \theta)$ 
8:      $\delta = R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})$ 
9:      $\theta \leftarrow \theta + \alpha^\theta \nabla_\theta \ln \pi(S \mid A, \theta) \hat{q}(S, A, \mathbf{w})$ 
10:     $\mathbf{w} \leftarrow \mathbf{w} + \alpha^w \delta \phi(S, A)$ 
11:     $A \leftarrow A'; S \leftarrow S'$ 
```

Actor-Critic Algorithm

Actor-Critic (continuing), for estimating $\pi\theta \approx \pi_*$

Require: a differentiable policy parameterization $\pi(a | s, \theta)$

Require: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Require: step sizes $\alpha^\theta > 0$, $\alpha^\mathbf{w} > 0$, $\alpha^{\bar{R}} > 0$

Require: Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g. to 0)

```
1: function VACCont( $\alpha^\theta, \alpha^\mathbf{w}, \alpha^{\bar{R}}$ )  
2:    $\bar{R} \leftarrow 0$  ▷ Initialize average reward  
3:   loop forever (for each time step)  
4:      $A \sim \pi(\cdot | S, \theta)$   
5:     Take action A, observe  $S', R$   
6:      $\delta \leftarrow R - \bar{R} + \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$   
7:      $\bar{R} \leftarrow \bar{R} + \alpha^{\bar{R}} \delta$   
8:      $\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \delta \nabla_\mathbf{w} \hat{v}(S_t, \mathbf{w})$   
9:      $\theta \leftarrow \theta + \alpha^\theta \delta \nabla_\theta \ln \pi(A_t | S_t, \theta)$   
10:     $S \leftarrow S'$ 
```

Bias in Actor-Critic Algorithms

- Approximating the policy gradient introduces bias
- A biased policy gradient may not find the right solution
 - e.g. if $\hat{q}(S, A, \mathbf{w})$ uses aliased features, can we solve gridworld example?
- Luckily, if we choose value function approximation carefully
Then we can avoid introducing any bias
i.e. We can still follow the **exact** policy gradient

Reducing Variance Using a Baseline

- We subtract a baseline function $b(s)$ from the policy gradient
- This can reduce variance, without changing expectation

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a | s, \theta) b(s)] &= \sum_{s \in \mathcal{S}} \mu(s) \sum_a \nabla_{\theta} \pi(a | s, \theta) b(s) \\ &= \sum_{s \in \mathcal{S}} \mu(s) b(s) \sum_a \nabla_{\theta} \pi(a | s, \theta) \\ &= 0\end{aligned}$$

- A good baseline is the state value function $b(s) = v_{\pi_{\theta}}(s)$
- So we can rewrite the policy gradient using the **advantage function** $A^{\pi_{\theta}}(s, a)$

$$\begin{aligned}A^{\pi_{\theta}}(s, a) &= q_{\pi_{\theta}}(s, a) - v_{\pi_{\theta}}(s) \\ \nabla_{\theta} \mathcal{L}(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(s | a, \theta) A^{\pi_{\theta}}(s, a)]\end{aligned}$$

Estimating the Advantage Function (1)

- The advantage function can significantly reduce variance of policy gradient
- So the critic should really estimate the advantage function
- For example, by estimating both $v_{\pi_{\theta}}(s)$ and $q_{\pi_{\theta}}(s, a)$
- Using two function approximators and two parameter vectors,

$$\begin{aligned}\hat{v}(s, \mathbf{w}_1) &\approx v_{\pi_{\theta}}(s) \\ \hat{q}(s, a, \mathbf{w}_2) &\approx q_{\pi_{\theta}}(s, a) \\ A(s, a) &= \hat{q}(s, a, \mathbf{w}_2) - \hat{v}(s, \mathbf{w}_1)\end{aligned}$$

- And updating both value functions by e.g. TD learning

Estimating the Advantage Function (2)

- For the true value function $v_{\pi_{\theta}}(s)$, the TD error $\delta^{\pi_{\theta}}$

$$\delta^{\pi_{\theta}} = R + \gamma v_{\pi_{\theta}}(s') - v_{\pi_{\theta}}(s)$$

- is an unbiased estimate of the advantage function

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}} [\delta^{\pi_{\theta}} \mid s, a] &= \mathbb{E}_{\pi_{\theta}} [r + \gamma v_{\pi_{\theta}}(s') \mid s, a] - v_{\pi_{\theta}}(s) \\ &= q_{\pi_{\theta}}(s, a) - v_{\pi_{\theta}}(s) \\ &= A^{\pi_{\theta}}(s, a)\end{aligned}$$

- So we can use the TD error to compute the policy gradient

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a \mid s, \theta) \delta^{\pi_{\theta}}]$$

- In practice we can use an approximate TD error

$$\delta_{\mathbf{w}} = r + \gamma \hat{v}(s', \mathbf{w}) - \hat{v}(s, \mathbf{w})$$

- This approach only requires one set of critic parameters $\mathbf{w}$

Critics at Different Time-Scales

- Critic can estimate value function $\hat{v}(s, \mathbf{w})$ from many targets at different time-scales From last lecture:
 - For MC, the target is the return G_t

$$\Delta\theta = \alpha (\textcolor{red}{G}_t - \hat{v}(s, \mathbf{w})) \phi(s)$$

- For TD(0), the target is the TD target $r + \gamma v(s')$

$$\Delta\theta = \alpha (\textcolor{red}{r} + \gamma \textcolor{red}{v}(\textcolor{red}{s}') - \hat{v}(s, \mathbf{w})) \phi(s)$$

- For forward-view $TD(\lambda)$, the target is the λ -return G_t^λ

$$\Delta\theta = \alpha \left(\textcolor{red}{G}_t^\lambda - \hat{v}(s, \mathbf{w}) \right) \phi(s)$$

- For backward-view $TD(\lambda)$, we use eligibility traces

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}) - v(s_t)$$

$$Z_t = \gamma \lambda Z_{t-1} + \phi(s_t)$$

$$\Delta\theta = \alpha \delta_t Z_t$$

Actors at Different Time-Scales

- The policy gradient can also be estimated at many time-scales

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)]$$

- Monte-Carlo policy gradient uses error from complete return

$$\Delta \theta = \alpha (G_t - \hat{v}(s_t, \mathbf{w})) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$$

- Actor-critic policy gradient uses the one-step TD error

$$\Delta \theta = \alpha (r + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w})) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$$

Policy Gradient with Eligibility Traces

- Just like forward-view TD(λ), we can mix over time-scales

$$\Delta \theta = \alpha \left(G_t^\lambda - \hat{v}(s_t, \mathbf{w}) \right) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$$

- where $G_t^\lambda - \hat{v}(S_t, \mathbf{w})$ is a biased estimate of advantage function
- Like backward-view TD(λ), we can also use eligibility traces
 - By equivalence with TD(λ), substituting $\phi(s) = \nabla_{\theta} \ln \pi_{\theta}(s, a)$

$$\begin{aligned}\delta &= r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \gamma \hat{v}(s_t, \mathbf{w}) \\ Z_{t+1} &= Z_t + \nabla_{\theta} \ln \pi_{\theta}(s, a) \\ \Delta \theta &= \alpha \delta Z_t\end{aligned}$$

- This update can be applied online, to incomplete sequences

Outline

- 1 Recap
- 2 Introduction
 - Examples
 - Policy Search
- 3 Monte-Carlo Policy Gradient
 - Likelihood Ratios
 - Policy Gradient Theorem
- 4 Actor-Critic Policy Gradient
 - Compatible Function Approximation
 - Advantage Function Critic
 - Eligibility Traces
- 5 Summary and Conclusion

Summary of Policy Gradient Algorithms

- The policy gradient has many equivalent forms

$\nabla_{\theta} \mathcal{L}(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a s, \theta) G_t]$	REINFORCE
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a s, \theta) \hat{q}(s, a, \mathbf{w})]$	Q Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a s, \theta) A(s, a, \mathbf{w})]$	Advantage Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a s, \theta) \delta]$	TD Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi(a s, \theta) \delta Z]$	TD(λ) Actor-Critic

- Each leads a stochastic gradient ascent algorithm
- Critic uses policy evaluation (e.g. MC or TD learning) to estimate $q_{\pi}(s, a)$, $A^{\pi}(s, a)$, or $v_{\pi}(s)$

Lecture Summary

- Policy Gradient Methods
 - Policy Gradient and REINFORCE algorithm
 - Actor-Critic Algorithms